

14

- a No funciona polimórficamente porque enivar () no es virtual, así que al usar el access por puntero se ejecuta el método de la clase base.
- b virtual en la línea 5
- c No, porque con virtual en la clase base las clases derivadas pueden sobrescribir sin problemas.
- d Memory leak al no despejar el heap para las clases derivadas
- e No porque utiliza la dirección real de e, por lo que trabaja sobre ese objeto.

M1 a A construido

B construido

C construido

~C destruido

~B destruido

~A destruido

b A * obj → objeto C

└ A: datos → int [5]

└ B: vals → double [3]

└ C: buf → char [10]

c Solo se llamaría el destructor ligado a A, quedando sin liberar vals y buf (B y C)

d Se construye en orden A, B y C; y se destruye en orden inverso, cumpliendo el principio LIFO

M5: a Destructor figura, no es un destructor virtual, por lo que no puede limpiar la memoria por sí solo.

```
virtual ~figura() {  
    delete [] coords;  
    coords = nullptr;  
}
```

b Destructor ~Canvas(), limpia el arreglo pero no los elementos creados dentro de él.

```
delete figuras[i];  
figuras[i] = nullptr;
```

c Main, new FiguraColor(2, 2) se pierde porque f cambia de dirección antes de hacer delete

```
delete f;  
f = nullptr;
```